LAB SESSION 7

Lab Name: Reliability, Dependability & Security Engineering

Project: MediScript-E — Digital Healthcare Platform

Objectives

- Identify system risks and failure scenarios in MediScript-E
- Design reliable and secure system components
- Perform threat analysis and define security controls

Theory

Dependability includes:

- **Reliability:** System performs correctly over time
- **Availability:** System is accessible when needed
- **Security:** System is protected from unauthorized access and misuse
- **Maintainability:** System can be updated and repaired efficiently

Security engineering protects systems from misuse and attacks. In a healthcare platform like MediScript-E, security is critical because sensitive medical data, personal information, and authentication credentials are involved.

Task 1: Failure Scenario Identification

Failure ID	Failure Scenario	Component Affected	Impact	Likelihood
FS-01	Database connection timeout	Prisma / Supabase PostgreSQL	High - All features fail	Medium
FS-02	Email service (SMTP) unavailable	Nodemailer / Gmail SMTP	High - Verification, OTP, reminders fail	Low
FS-03	Supabase Storage unavailable	Medical Vault	Medium - File upload/download fails	Low
FS-04	JWT secret key compromised	NextAuth session management	Critical - All sessions compromised	Very Low
FS-05	Vercel serverless function timeout	API Routes	High - API requests fail	Low
FS-06	Invalid/expired email verification token	Auth module	Medium - User cannot verify email	Medium

Failure ID	Failure Scenario	Component Affected	Impact	Likelihood
FS-07	OTP brute force attack	2FA module	High - Account takeover possible	Medium
FS-08	Unauthorized API access without session	All protected API routes	Critical - Data breach	Low
FS-09	File upload with malicious content	Medical Vault	High - Server compromise	Low
FS-10	Admin accidentally deletes critical user	Admin module	Medium - Data loss	Low
FS-11	Cron job fails to execute	Medicine reminder system	Low - Reminders not sent	Medium
FS-12	OAuth provider (Google/GitHub) downtime	Authentication module	Medium - OAuth login unavailable	Very Low
FS-13	SQL injection via user input	Database layer	Critical - Data breach	Low
FS-14	Session token theft (XSS)	Frontend / NextAuth	Critical - Account hijacking	Low
FS-15	Concurrent appointment booking conflict	Appointment module	Medium - Double booking	Medium

Task 2: Reliability Targets

Component	Reliability Target	Measurement	Current Implementation
Overall System Uptime	99.9%	Monthly availability	Vercel production deployment
Database Availability	99.95%	Monthly uptime	Supabase managed PostgreSQL
API Response Time	< 2 seconds	95th percentile	Vercel serverless functions
Email Delivery	> 95% success rate	Per email sent	Gmail SMTP via Nodemailer
Authentication Success	> 99% for valid credentials	Per login attempt	NextAuth with bcrypt
File Upload Success	> 98%	Per upload attempt	Supabase Storage
OTP Delivery	> 95% within 60 seconds	Per OTP request	Gmail SMTP
Cron Job Execution	> 99% on schedule	Per scheduled run	External cron service

Reliability Design Decisions

Decision	Justification
Supabase managed PostgreSQL	Automatic backups, high availability, managed SSL
Vercel serverless deployment	Auto-scaling, zero-downtime deployments
Connection pooling (pg Pool, max: 20)	Prevents database connection exhaustion
JWT session strategy	Stateless, no server-side session storage needed
OTP expiry (10 minutes)	Balances security and usability
Email verification token expiry (24 hours)	Sufficient time for user action

---

Task 3: Threat Analysis

3.1 Authentication Threats

Threat ID	Threat	Attack Vector	Likelihood	Impact
T-01	Brute force password attack	Repeated login attempts	Medium	High
T-02	Credential stuffing	Using leaked credentials from other sites	Medium	High
T-03	OTP brute force	Repeated OTP guesses	Medium	High
T-04	Session token theft	XSS attack stealing JWT	Low	Critical
T-05	OAuth token interception	Man-in-the-middle attack	Very Low	Critical
T-06	Email verification bypass	Manipulating verification URL	Low	High
T-07	2FA bypass via direct API call	Calling signIn with fake bypass params	Low	Critical

3.2 Data Threats

Threat ID	Threat	Attack Vector	Likelihood	Impact
T-08	SQL injection	Malicious input in forms	Low	Critical
T-09	Unauthorized data access	Missing auth checks on API routes	Low	Critical
T-10	Insecure file upload	Uploading malicious files	Low	High
T-11	Data exposure via API	API returning sensitive fields	Medium	High
T-12	CSRF attack	Forged cross-site requests	Low	High

3.3 Infrastructure Threats

Threat ID	Threat	Attack Vector	Likelihood	Impact
T-13	Environment variable exposure	Leaked .env file in repository	Low	Critical
T-14	Database credential exposure	Hardcoded credentials in code	Very Low	Critical
T-15	DDoS attack	Flooding API endpoints	Low	High

Task 4: Security Controls

4.1 Authentication Security Controls

Threat	Security Control	Implementation in MediScript-E
T-01 Brute force	Password hashing with bcryptjs	`bcrypt.hash(password, 10)` — 10 salt rounds
T-02 Credential stuffing	Email verification requirement	`emailVerified: false` blocks login until verified
T-03 OTP brute force	OTP expiry + single use	OTP expires in 10 minutes, cleared after use
T-04 Session theft	JWT with secure secret	`NEXTAUTH_SECRET` environment variable, HttpOnly cookies
T-05 OAuth interception	HTTPS enforced	Vercel enforces HTTPS on all routes
T-06 Verification bypass	Token expiry + unique token	`verificationExpires` checked, token cleared after use
T-07 2FA bypass	OTP null check before bypass	`twoFactorCode !== null` verified before session creation

4.2 Data Security Controls

Threat	Security Control	Implementation in MediScript-E
T-08 SQL injection	Prisma ORM parameterized queries	All DB operations via Prisma — no raw SQL
T-09 Unauthorized access	Role-based access control (RBAC)	`getServerSession()` checked on every protected API route
T-10 Malicious file upload	File type and size validation	Validated before Supabase Storage upload
T-11 Data exposure	Selective field queries	Prisma `select` used to return only required fields
T-12 CSRF	NextAuth CSRF protection	Built-in CSRF token in NextAuth

4.3 Infrastructure Security Controls

Threat	Security Control	Implementation in MediScript-E
T-13 Env variable exposure	`.env` in `.gitignore`	`.env` never committed to repository
T-14 Hardcoded credentials	Environment variables only	All secrets in `.env` / Vercel environment variables
T-15 DDoS	Vercel rate limiting	Vercel platform-level DDoS protection

Security Design Document

Security Architecture Overview

```
[Security Architecture Diagram - MediScript-E]
```

Security Layers

```
Layer 1: Network Security
├── HTTPS enforced (Vercel)
├── SSL/TLS database connection (sslmode=no-verify)
└── Supabase Storage secure URLs

Layer 2: Authentication Security
├── bcryptjs password hashing (10 salt rounds)
├── Email verification (24-hour token expiry)
├── JWT session management (30-day expiry)
├── OAuth via Google & GitHub (auto-verified)
└── 2FA Email OTP (10-minute expiry, single use)

Layer 3: Authorization Security
├── Role-Based Access Control (PATIENT / DOCTOR / ADMIN)
├── getServerSession() on all protected API routes
├── Admin cannot delete own account
└── Patients can only access their own data

Layer 4: Input Security
├── Prisma ORM (prevents SQL injection)
├── Input validation on all API routes
├── File type and size validation for uploads
└── Email format validation on registration

Layer 5: Data Security
├── Passwords never stored in plaintext
├── OTP cleared from database after use
```

```
  ├── Verification tokens cleared after use
  ├── Selective field queries (no sensitive data exposure)
```

RBAC Matrix

Action	Public	Patient	Doctor	Admin
View landing page	✅	✅	✅	✅
Register / Login	✅	✅	✅	✅
Book appointment	×	✅	×	×
Manage appointments	×	×	✅	×
Issue prescription	×	×	✅	×
View prescription	×	✅	✅	×
Upload medical document	×	✅	×	×
Set medicine reminder	×	✅	×	×
View admin dashboard	×	×	×	✅
Delete users	×	×	×	✅
View all appointments	×	×	×	✅
Toggle 2FA	×	✅	✅	×

Key Findings / Learning Outcomes

- Identified **15 failure scenarios** and **15 security threats** specific to MediScript-E
- Understood that healthcare platforms require multiple security layers due to sensitive medical data
- Learned that **Prisma ORM** inherently prevents SQL injection through parameterized queries
- Recognized that **2FA** significantly reduces account takeover risk even when passwords are compromised
- Understood the importance of **environment variables** for protecting credentials in production
- Designed a comprehensive **RBAC matrix** ensuring each role has access only to relevant features
- Learned that **SSL/TLS** at the database connection level is critical for data-in-transit protection